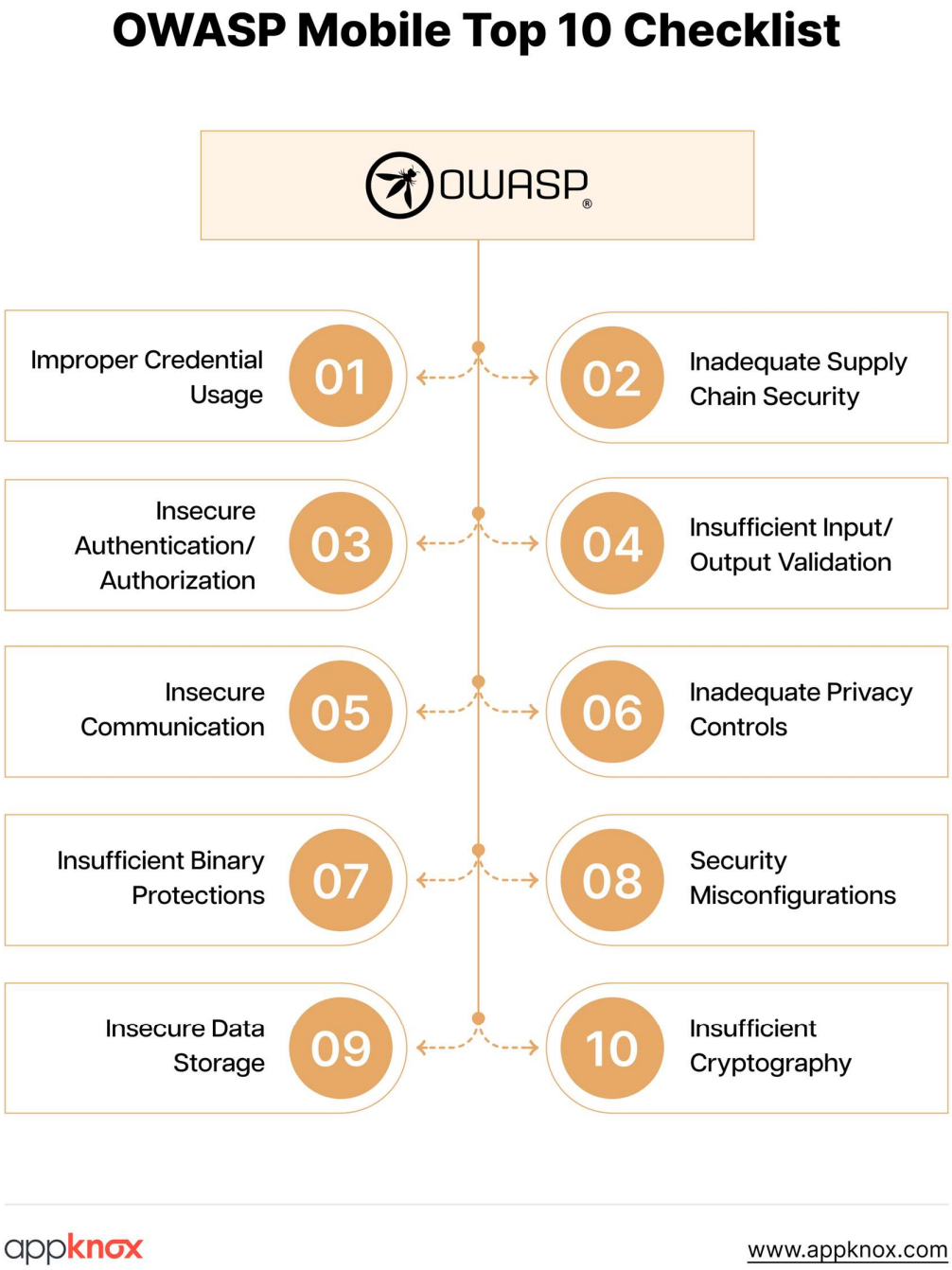
TASK: 25

OWASP Mobile Top 10

Gabbani Surendra

What is OWASP Mobile top 10?

The OWASP Mobile Top 10 is a list created by the Open Web Application Security Project (OWASP) that highlights the top 10 most critical security risks found in mobile applications (both Android and iOS.



Here's a brief overview of the OWASP Mobile Top 10:

1. M1: Improper Credential Usage

Example: Hard-coded credentials in the mobile app (API keys, secret tokens) or storing user credentials insecurely on the device.
Prevention:

- Never embed credentials in source code or resources; rotate API keys. Medium+2security.docs.wso2.com+2
- Securely store credentials (e.g., OS keychain, secure vault) and protect them both at rest and in transit. Cobalt+1
- Implement strong authentication (multi-factor), and enforce credential policy.

2. M2: Inadequate Supply Chain Security

Example: Using a third-party SDK library that has a known vulnerability or malicious code is injected via a compromised dependency.
Prevention:

- Audit and monitor all external dependencies (libraries, SDKs, build tools) for vulnerabilities (CVEs). Cobalt+1
- Use secure build, code signing, and distribution pipelines.
- Keep dependencies up to date; remove unused ones.

3. M3: Insecure Authentication / Authorization

Example: The app allows access to user data without properly verifying the user's identity.Weak session management — e.g., no token expiry or reuse of old tokens.Bypassing login using modified API calls or intercepting requests.

Prevention:

- Implement secure authentication mechanisms (OAuth 2.0, JWT).
- Enforce session timeouts and token expiration.
- Verify user permissions on both client and server sides.
- Use backend validation — never trust client-side authorization.

4. M4: Insufficient Input/Output Validation

Example: User-input fields in a mobile app accept arbitrary input which leads to SQL injection, or output is rendered without sanitization leading to XSS-type issues. Strobes Security+1

Prevention:

- Always validate and sanitize user input (length, format, allowed characters).
- Sanitize output when rendering data to avoid injection or code execution. Strobes Security+1
- Use parameterized queries or safe APIs rather than concatenating user input into commands or queries.

5. M5: Insecure Communication

Example: The mobile app sends sensitive data (like login credentials, personal info) over HTTP or uses deprecated TLS versions, allowing interception ("man-in-the-middle").

Prevention:

- Use strong TLS/SSL (latest versions), enforce certificate pinning if applicable. Medium+1
- Do not transmit sensitive data in plaintext; ensure endpoints are secure.
- Verify server certificates and hostnames correctly.

6. M6: Inadequate Privacy Controls

Example: Collecting sensitive user data (contacts, location, microphone access) unnecessarily. Failing to comply with privacy laws like GDPR. Storing personal data without encryption or user consent.

Prevention:

- Follow the principle of least privilege — request only necessary permissions.
- Clearly inform users about data collection and obtain consent.
- Use strong data protection and anonymization where possible.
- Regularly audit data collection and retention policies.

7. M7: Insecure Data Storage

Example: Sensitive data stored in unprotected shared preferences or SQLite databases.Data visible after app uninstall or accessible via backup tools.
Cached files containing user details.
Prevention:

- Encrypt all sensitive local data using secure APIs (Android Keystore, iOS Keychain).
- Avoid storing passwords or tokens on the device.
- Use app sandboxing to isolate data.
- Delete temporary or cached sensitive files regularly.

M8: Security Misconfiguration

Example:
Debug mode left enabled in production builds.
Misconfigured permissions or exported components.
Default credentials and unused services left active.
Prevention:

- Disable debugging, logging, and test endpoints before release.
- Review and harden app manifest/config files.
- Use secure build configurations (ProGuard, code obfuscation).
- Regularly scan for misconfigurations using mobile security tools (MobSF, QARK).

9. M9: Insecure Data Storage

Example: App writes sensitive user data (passwords, session tokens, PII) to unencrypted local storage or database on the device which can be extracted if device is compromised.
Prevention:

- Store as little data locally as possible. When storing, encrypt data with strong algorithms. security.docs.wso2.com+1
- Use platform-secure storage APIs (Keychain on iOS, Android Keystore) rather than generic file storage.
- Consider data lifecycle: delete when no longer needed; handle lost/stolen device scenarios.

10. M10: Insufficient Cryptography

Example: Using weak encryption algorithms (e.g., MD5, SHA1, RC4), or rolling own custom crypto rather than vetted libraries, leading to weak protection of stored or transmitted data.
Prevention:

- Use modern, approved cryptographic algorithms and libraries (AES-GCM, SHA-256/512, etc.). OWASP
- Ensure secure key generation and storage, avoid custom/undocumented cryptography.
- Make sure encryption is applied correctly (both rest & transit) and keys are protected.

THANK YOU